

Agentic AI Engineer — Portfolio Build Specification

Owner: You **Goal:** Land first remote freelance contract within 8–10 weeks of starting **Strategy:** Eval harness as centerpiece. Every project ships with metrics, a video, and a written case study. Outreach begins after Project 1 + 2 are presentable — not after all five are done.

Portfolio Thesis (read this before building anything)

You are not selling "I can build agents." Everyone claims that. You are selling:

"I can build agentic systems for your unique domain, and prove they work — with numbers."

Every project below exists to support that one sentence. The eval harness is the spine; the other four projects are proof it works in radically different contexts. When a client with a weird use case asks "but will this work for *us*?", your portfolio answers before you speak.

The universal deliverable set (every project ships all five):

1. Working deployed demo (or recorded end-to-end run if deployment is impractical)
2. Clean public repo with a README that a senior engineer respects
3. Architecture diagram (one page, honest about tradeoffs)
4. Metrics table — accuracy/eval scores, cost per run, latency
5. 2–4 minute video walkthrough + written case study (problem → approach → tradeoffs → numbers → what failed)

If a project is missing any of these five, it is not done.

Project 1 — "Proving Ground": Domain-Adaptive Agent Eval Harness

Role in portfolio: Centerpiece + your open-source play. This is the project you lead every pitch with. **Build time:** 3–4 weeks (core in 2, domain packs in 1–2) **Positioning line:** *"Plug in any agent, any domain — get a reliability scorecard in an afternoon."*

What exactly to build

A framework that takes (a) an agent system, (b) a domain-specific golden dataset, and (c) a metric configuration — and produces a full evaluation report with regression tracking.

Core engine (weeks 1–2)

- **Golden dataset toolkit.** A structured format for test cases: input, expected outcome, acceptable-variation rules, difficulty tag, failure-category tag. Include a semi-automated dataset builder: feed it raw domain documents/conversations, it drafts candidate test cases, a human approves/edits. This human-in-the-loop dataset creation is itself a selling point — clients never have golden datasets.
- **Trajectory capture.** Record the agent's full run, not just the final answer: every tool call, every intermediate reasoning step, every retry. Store as structured traces (JSON) so they're replayable and diffable.
- **Metric layer with three tiers:**
 - *Deterministic checks* — exact match, schema validation, tool-call correctness (did it call the right tool with the right args), constraint satisfaction (e.g., "never emailed without approval").
 - *Statistical/retrieval metrics* — precision/recall against golden citations, latency percentiles (p50/p95), cost per task.
 - *LLM-as-judge* — faithfulness, helpfulness, tone, with a critical requirement: **calibrate the judge**. Hand-label 50 cases yourself, measure judge agreement with your labels, report that agreement number. This single step puts you ahead of nearly everyone using LLM judges blindly.
- **Regression runner.** One command re-runs the full suite against a new agent version and produces a diff report: what improved, what regressed, per failure category. Store history so you can chart eval scores over versions.
- **Report generator.** HTML/markdown scorecard: overall pass rate, per-category breakdown, cost and latency, worst 10 failures with full trajectories attached, trend chart across versions.

Domain packs (weeks 3–4) — the "undeniable" part

Ship the same harness applied to **three wildly different domains**, each as a worked example in the repo:

1. **Legal/contract extraction** — golden set of ~50 clauses to extract from real public contracts (SEC EDGAR exhibits are free and messy). Metrics: extraction accuracy, citation faithfulness, hallucinated-clause rate.
2. **Customer-support agent** — golden set of ~50 support scenarios (build from a public dataset or synthesize + hand-review). Metrics: resolution correctness,

escalation-when-required rate, forbidden-action rate (refunds over limit, promises it can't keep).

3. **Financial document QA** — ~50 questions over earnings reports/10-Ks. Metrics: numerical accuracy (exact figures, this exposes agents brutally), source-grounding rate.

Each pack must include: the golden dataset, the metric config, a baseline agent, the full eval report, and a short writeup: *"What the harness caught in this domain that a demo would have hidden."* That sentence structure is your marketing.

Scope discipline — do NOT build

- No web UI beyond the generated HTML report. CLI + config files is fine and more credible.
- No support for every framework. Adapters for two (e.g., raw API calls + one popular agent framework) with a documented interface for others.
- No hosted SaaS version. Resist this completely.

Definition of done

- A stranger can clone the repo, run one command, and reproduce the legal-domain eval report.
- README shows the scorecard image in the first screen-scroll.
- You've published one deep-dive post: "I evaluated the same agent architecture across 3 domains. Here's where it silently failed."
- Calibrated judge agreement number is published (even if it's 82%, honesty sells).

Case study angle

"95% of agent portfolios show demos. Demos hide failures. Here's a framework that finds them — and here's what it found."

Project 2 — Autonomous Multi-Agent Business Workflow

Role in portfolio: The "this is what you'd hire me to build" project. Directly mirrors paid work.

Build time: 3 weeks **Positioning line:** *"An agent team that runs a real revenue process end-to-end — with approval gates, failure recovery, and a cost meter."*

What exactly to build

Pick one concrete process and go deep. Recommended: inbound lead → qualified, personalized outreach. (Alternative if you prefer: invoice intake → validation → dispute flagging → payment scheduling. Choose one, not both.)

The pipeline (each stage is a distinct agent or agent role)

1. **Intake agent** — receives a lead (webhook or email parse): name, company, message. Normalizes into a structured record.
2. **Research agent** — enriches the lead: company website scrape, recent news search, size/industry inference, tech-stack hints. Must produce *cited* research notes (every claim links a source) — this is where your harness's faithfulness metric plugs in.
3. **Qualification agent** — scores the lead against a configurable ICP (ideal customer profile) rubric. Outputs score + reasoning + confidence. Low-confidence cases route to human review, not silent guesses.
4. **Outreach drafting agent** — writes a personalized email grounded in the research notes. Hard constraint: every personalized claim must trace to a research citation. No hallucinated flattery.
5. **Human approval gate** — drafts land in a review queue (a simple web page or even a Slack message with approve/reject buttons). **Nothing sends without approval.** Make this gate prominent in your demo — it's what makes business buyers trust the system.
6. **Execution agent** — on approval: sends via Gmail API, logs to a real CRM (HubSpot free tier), creates a follow-up task on the calendar.

Engineering requirements (this is what separates it from a toy)

- **Durable state.** Every lead's progress persisted; the system resumes after a crash mid-pipeline. Kill the process during a demo run and show it recover — put that in the video.
- **Failure handling per stage.** Retries with backoff, dead-letter queue for permanently failed leads, alerting. Document the failure taxonomy: transient API errors vs. model refusals vs. bad data.
- **Cost meter.** Per-lead cost tracked and displayed: tokens per stage, total dollars. Show the number in the dashboard: "This lead cost \$0.14 to process."
- **Observability.** Every run fully traced (tool calls, prompts, outputs). One click from "lead #47 failed" to the exact trace.
- **Evals via Project 1.** Golden set of ~30 test leads (mix of great fits, obvious rejects, ambiguous cases, adversarial/spam). Published scorecard: qualification accuracy, research citation faithfulness, forbidden-action rate (did it ever attempt to send without approval — must be 0).

Scope discipline — do NOT build

- No multi-channel outreach (email only; no LinkedIn automation — it violates ToS and looks bad).
- No fancy frontend. A functional review queue beats a pretty dashboard.
- No support for arbitrary workflows. One workflow, done excellently.

Definition of done

- Live demo processes a fresh lead end-to-end in under 3 minutes on video, including the human approval moment and the crash-recovery moment.
- Eval scorecard published: qualification accuracy $\geq$ your stated bar on the golden set, forbidden-action rate = 0.
- Case study includes the cost math: "at \$0.14/lead and X leads/month, this replaces Y hours of SDR research time."

Case study angle

"Businesses don't buy agents; they buy reliable processes. Here's the engineering that makes an agent trustworthy enough to touch a real CRM."

Project 3 — Production RAG with a Published Failure Analysis

Role in portfolio: Depth proof in your chosen vertical. Fastest to finish. Consider building this FIRST as a warm-up that also feeds Project 1's financial domain pack. **Build time:** 2 weeks

Positioning line: *"RAG over genuinely messy documents — with an eval suite and an honest list of where it breaks."*

What exactly to build

Corpus: pick real, messy, free documents in one vertical. Recommended: 20–30 SEC 10-K filings (tables, footnotes, cross-references — brutal and impressive) or Indian regulatory filings if you want a Indian-market angle. Not Wikipedia. Not clean PDFs.

The system

- **Ingestion pipeline that handles the mess:** table extraction that preserves structure, footnote linking, section-aware chunking (respect document hierarchy, don't split mid-table), metadata extraction (company, fiscal year, section type) for filtered retrieval.
- **Retrieval with at least two strategies compared:** e.g., hybrid (dense + keyword) vs. dense-only, with and without a reranker. You will publish the comparison numbers — the comparison IS the content.
- **Grounded generation:** every answer cites specific chunks; a verification pass checks each numerical claim against its cited source before answering. Refuses when retrieval confidence is low — measure and report the refusal rate and whether refusals were correct.
- **Eval suite (via Project 1):** 60–100 question golden set you hand-build, spanning: simple lookup, cross-section synthesis, table-reading, questions whose answer is NOT in the corpus (tests hallucination), and ambiguous questions (tests clarification behavior). Metrics: answer accuracy, retrieval hit rate, citation faithfulness, hallucination rate on unanswerable questions, cost + latency per query.

The differentiator: the failure analysis

Publish a section most people hide: **"The 12 questions my system got wrong, and why."**

Categorize the failures (retrieval miss vs. table misread vs. synthesis error vs. judge disagreement). Show one fix and its measured effect on the scorecard. This honesty converts senior buyers harder than a 99% claim they won't believe.

Scope discipline — do NOT build

- No chat UI with history/streaming polish. A minimal query interface is fine.
- No 10,000-document corpus. 20–30 hard documents, deeply handled.
- No agentic-RAG complexity in v1 (multi-hop planning etc.) — note it as future work with a hypothesis of what it would fix.

Definition of done

- Scorecard published with the two-strategy comparison.
- Failure analysis section written.
- Feeds its golden set into Project 1's financial domain pack (one dataset, two portfolio pieces).

Case study angle

"Anyone can build RAG that works on easy questions. Here's what it takes on real filings — measured, with the failures included."

Project 4 — Browser Agent for a Genuinely Boring, Genuinely Useful Task

Role in portfolio: The viral demo. Video of this working is your most shareable asset. **Build time:** 2–3 weeks **Positioning line:** *"An agent that operates a real browser through a 20+ step task — and knows when it's failing."*

What exactly to build

Pick one task with real economic value and visible difficulty. Recommended: competitor price & availability monitoring across 5–8 real e-commerce/SaaS pricing pages (respect robots.txt; choose sites that permit it, or use your own test properties plus permissive ones). Alternative: multi-portal form submission against sandbox/demo portals.

The system

- **Task runner:** given a target list, the agent navigates each site, handles cookie banners/popups/layout differences, locates the product/plan, extracts price + availability + plan details, and screenshots evidence for every extracted datum.
- **State verification loop — the hard, impressive part:** after every action, the agent verifies the expected state change actually happened (URL changed, element appeared, price visible) before proceeding. On mismatch: re-observe, re-plan, retry with a different strategy (different selector path, search box instead of nav menu), and after N failures, mark the site failed with a diagnostic — never silently return garbage.
- **Anti-fragility catalog:** deliberately test against the ugly cases — infinite-scroll pages, prices rendered in images, A/B-tested layouts, region popups — and document per-case handling or documented failure.
- **Output:** structured dataset (per site: price, plan, timestamp, evidence screenshot, confidence) + a daily diff report ("Competitor X dropped Pro plan price 12%").
- **Evals (via Project 1):** run the full target list 20 times across several days. Report: per-site success rate, extraction accuracy vs. hand-checked ground truth, mean steps per task, cost per run, and **self-awareness rate** — when the agent failed, what fraction of the time did it *know* it failed vs. return wrong data confidently. That last metric is rare and devastating in a good way.

The video (as important as the code)

One continuous, sped-up screen recording: agent works through 3 sites including one where something breaks and it visibly recovers. Overlay step counter and running cost. 90 seconds. This is your LinkedIn/X pinned post.

Scope discipline — do NOT build

- No general-purpose "do anything" browser agent. One task class, high reliability.
- No CAPTCHA circumvention, no login-wall evasion, no ToS-violating scraping. Say so explicitly in the README — it signals professionalism, not weakness.

Definition of done

- ≥90% success rate on the chosen site list across 20 runs, honestly reported per-site.
- Self-awareness metric published.
- The 90-second video exists and has been posted.

Case study angle

"Browser agents demo well and fail silently in production. Here's one that verifies every step and reports its own failures."

Project 5 — The Optimization Case Study: Same Quality, 75% Cheaper

Role in portfolio: The one that speaks directly to buyers already burned by LLM bills. Shortest build; highest conversion per hour spent. **Build time:** 1–2 weeks (reuses Project 2 or 3 as the subject) **Positioning line:** *"I cut this agent system's cost 75% and its latency 60% — eval scores unchanged. Here's exactly how."*

What exactly to build

Not a new system — a **public, rigorous optimization of one of your own projects** (Project 2 is the ideal subject). Structure it as a before/after experiment with the harness as referee.

The experiment protocol

1. **Baseline measurement.** Freeze the naive version. Full eval run: quality scores, cost per task (broken down per stage per model), latency p50/p95. Publish the baseline table.
2. **Apply optimizations one at a time, re-running evals after each** so every technique gets its own attribution:
 - *Model routing* — cheapest model that passes evals per stage (intake/normalization needs far less model than research synthesis). Show the routing table and the eval proof that quality held.
 - *Prompt caching* — restructure prompts so static context (instructions, schemas, few-shot examples) hits the cache; measure the cost delta.
 - *Prompt compression* — cut bloated instructions; show token counts before/after and that eval scores held (or where they dipped and you rolled back — include the rollback, it's credibility gold).
 - *Parallelization* — stages/tool-calls that were needlessly sequential; latency delta.
 - *Selective escalation* — cheap model first, escalate to frontier model only when confidence is low; report escalation rate and quality impact.
 - *Output schema tightening* — structured outputs over free text where possible; measure retry-rate reduction.
3. **Final table.** One chart: cost and latency per optimization step, with eval score line flat across the top. That chart is the whole pitch.

Deliverable is the writeup itself

Long-form post (this one is written content first, repo second): the protocol, the per-technique numbers, the one technique that *didn't* work and why, and a general checklist readers can apply. End with the offer, stated plainly: "I do this as a fixed-price audit for production agent systems."

Scope discipline

- Don't invent a strawman baseline. Optimize your real Project 2 — its naive version is naive honestly.
- Don't claim savings without the eval-scores-held proof. Cost cuts with quality proof is the entire product.

Definition of done

- Per-technique attribution table published.
- The flat-quality/falling-cost chart exists.
- The post ends with the audit offer and your contact.

Case study angle

"Everyone's agent works. Nobody's agent is affordable. Here's a measured protocol that fixed that — quality verified at every step."

Build Order & Timeline

Weeks	Focus	Outreach status
1–2	Project 3 (RAG) — fast win, feeds harness dataset	Set up profiles, warm your network
3–5	Project 1 (Harness) core + financial pack (from P3)	Soft outreach begins — share P3 writeup
6–8	Project 2 (Workflow)	Active pitching with P1+P2+P3; offer the reliability audit
9–10	Project 5 (Optimization, subject = P2)	Publish the cost post — this one travels
11–13	Project 4 (Browser agent) + remaining harness domain packs	Full outreach; the 90-sec video as pinned content

Non-negotiable rule: publish something every single week from week 2 — a writeup, a scorecard, a failure analysis, a video. The portfolio compounds through distribution, not completeness.

Anti-Slop Checklist (apply to every project before calling it done)

- Does the README show a *number* in the first screen? (score, cost, success rate)
- Is there at least one honestly reported failure with analysis?
- Could a senior engineer reproduce the headline result with one command?
- Is there a video under 4 minutes?
- Does the case study end with what you'd do differently?

- Is there anything in the repo you built because a tutorial did, rather than because the problem required it? Delete it.